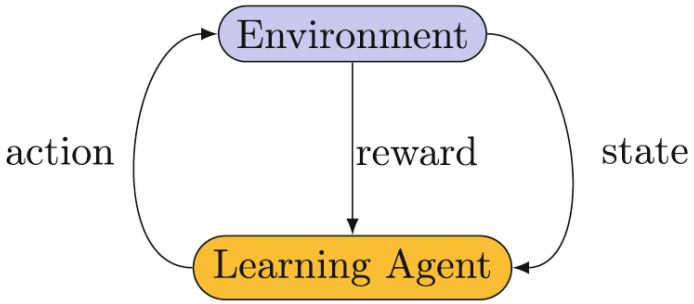
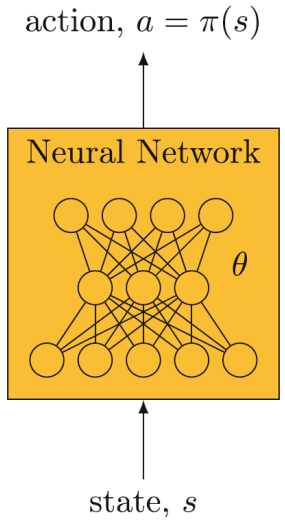
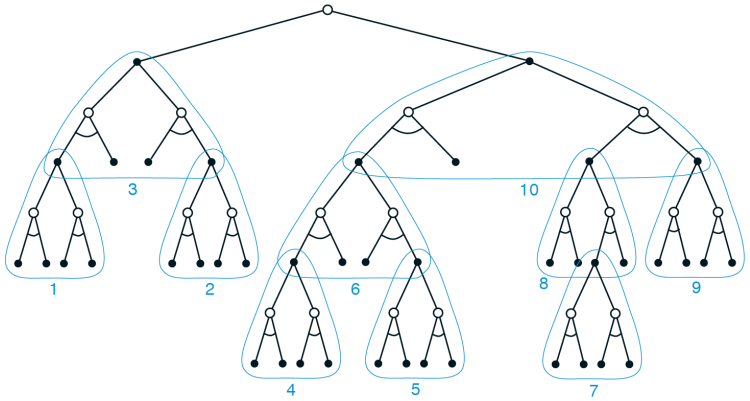
Dynamic Programming

Forest Agostinelli
University of South Carolina







Outline

- Review
- Policy Evaluation
- Policy Improvement
- Policy Iteration
  - Modified Policy Iteration
  - Value Iteration

Markov Decision Processes (MDPs)

- **States**
- **Actions**
- **Transition Probabilities:** $P(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a)$
  - Defines the dynamics of the MDP
- The state-transition probabilities can be obtained from the transition probabilities
  - $p(s'|s,a) = \sum_{r \in \mathcal{R}} p(s', r|s, a)$
- The expected reward can be obtained from the transition probabilities
  - $r(s,a) = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r|s, a) = \mathbb{E}[R_{t+1}|S_t = s, A_t = a]$
- For now, we assume the state and actions are discrete and finite, however, this restriction can be relaxed to be continuous and infinite

MDPs: Returns and Value

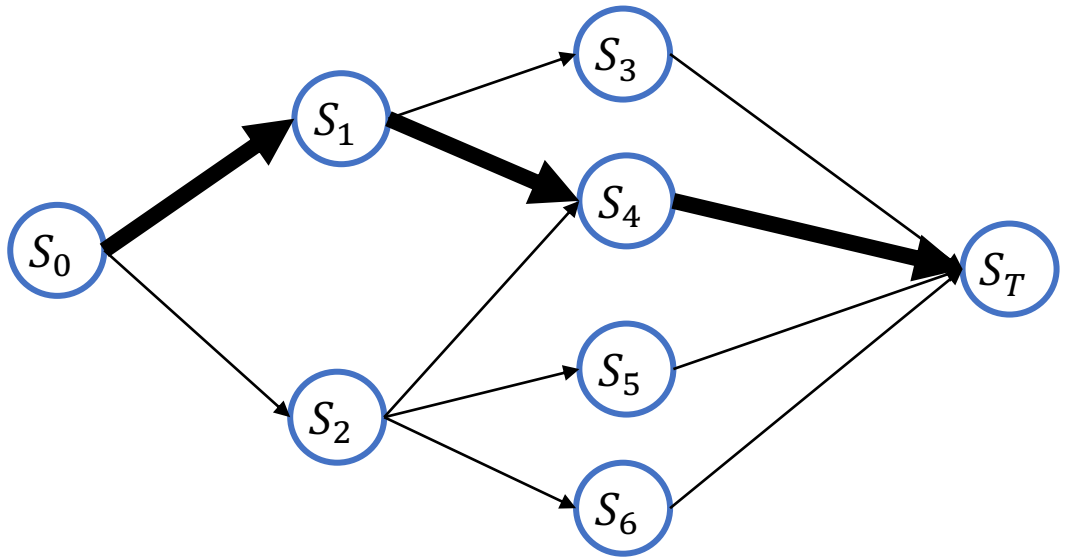
- **Return:** the sum of rewards after timestep t
  - $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \ldots$
  - We seek to maximize the expected return
- **State-value function**
  - $v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} \, | S_t = s]$
  - $v_*(s) = \max_\pi v_\pi(s)$
- **Action-value function**
  - $q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} \, | S_t = s, A_t = a]$
  - $q_*(s, a) = \max_\pi q_\pi(s, a)$
- Value functions are specific to a given policy π

Dynamic Programming

- Solves problems by recursively breaking them down into simpler subproblems
- Requires
  - Optimal substructure: Can construct an optimal solution from optimal solutions of subproblems

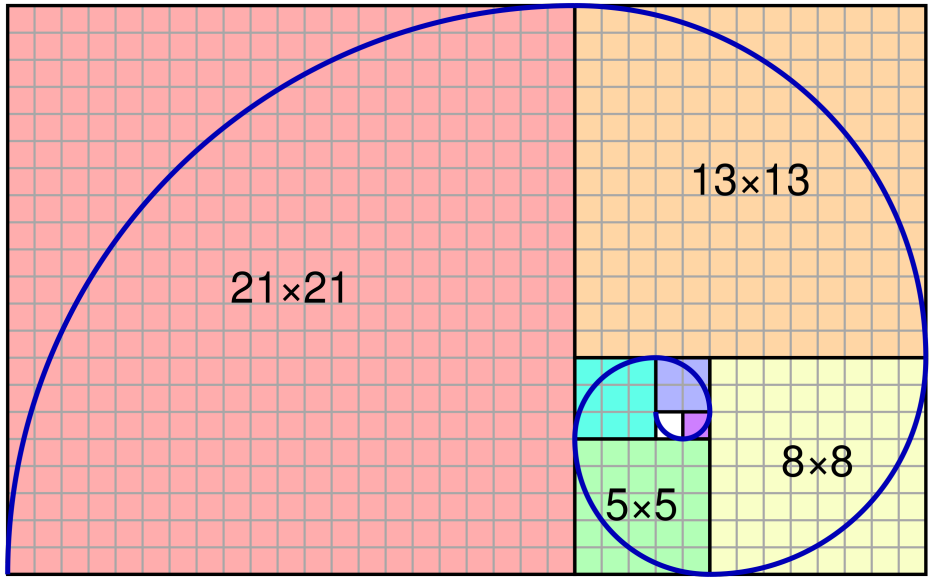
$$v_*(s) = \max_a(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_*(s'))$$

    - Principle of optimality
  - Overlapping subproblems: Solutions to subproblems are re-used
    - Value functions

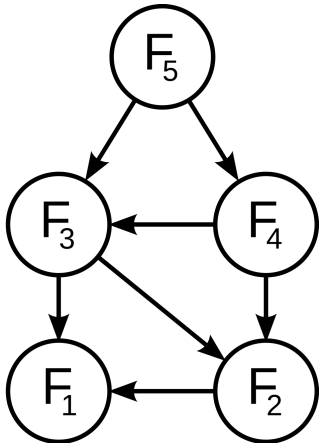


Dynamic Programming

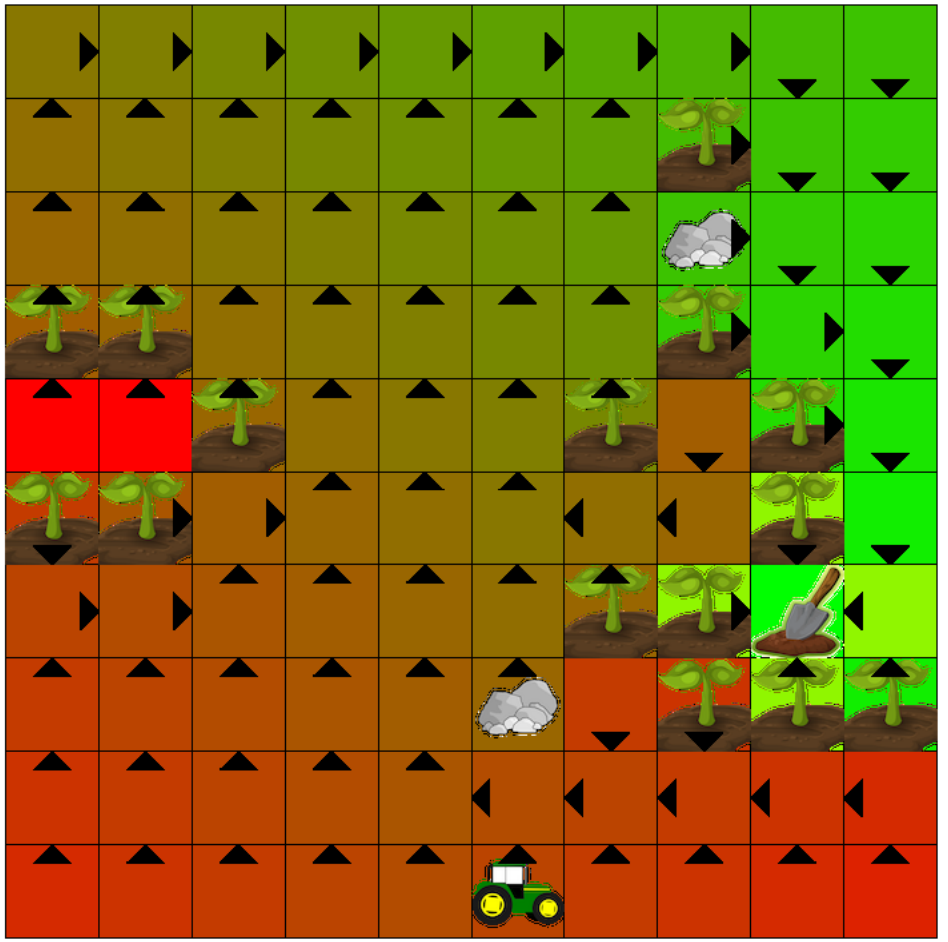
- Fibonacci sequence
- Scheduling
- Sequence alignment (DNA)
- AI Farm



By Jahobr - Own work, CC0, https://commons.wikimedia.org/w/index.php?curid=58460223



By en:User:Dcoatzee, traced by User:Stannered - en:Image:Fibonacci dynamic programming.png, Public Domain, https://commons.wikimedia.org/w/index.php?curid=3325402



Dynamic Programming: History

- The term *dynamic programming* was originally used in the 1940s by Richard Bellman to describe the process of solving problems where one needs to find the best decisions one after another.
- By 1953, he refined this to the modern meaning, referring specifically to nesting smaller decision problems inside larger decisions.
- The word *dynamic* was chosen by Bellman to capture the time-varying aspect of the problems, and because it sounded impressive.
- The word *programming* referred to the use of the method to find an optimal *program.*

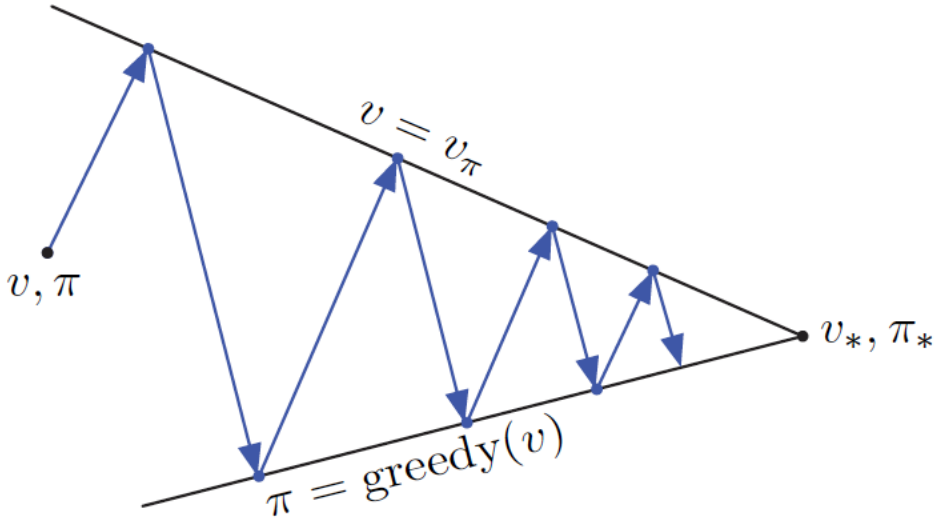
From: https://en.wikipedia.org/wiki/Dynamic_programming

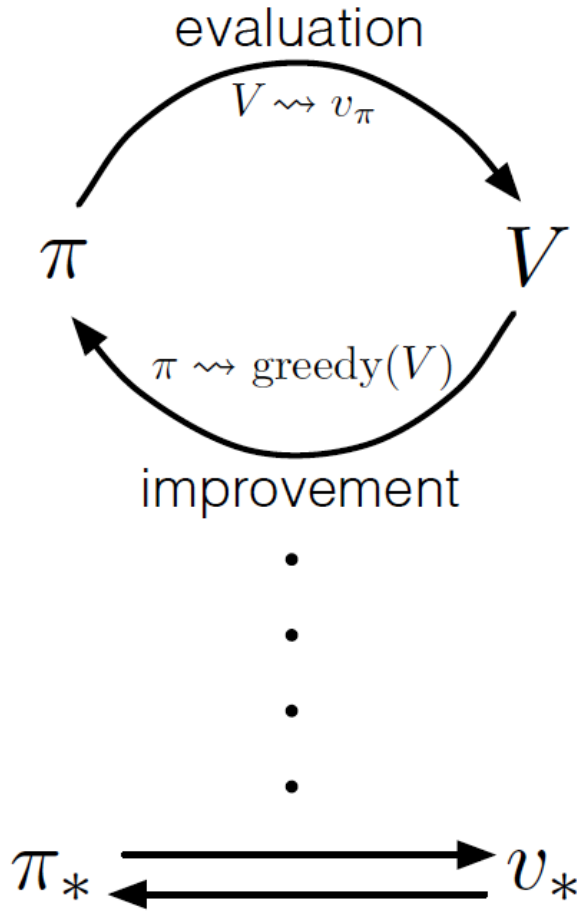
Dynamic Programming

- We will use it to evaluate a policy and compute an optimal policy given a perfect model an MDP
  - $p(s', r|s, a)$
- Foundational for reinforcement learning
- Using dynamic programming, we will do policy iteration by iterating between policy evaluation and policy improvement

Generalized Policy Iteration

- **Policy Evaluation:** Estimate the expected future reward when following policy π
- **Policy Improvement:** Improve policy π so that it obtains a greater expected future reward
- We can obtain an optimal policy by iterating between policy evaluation and policy improvement





Dynamic Programming: Applications

- We assume that we are given a model of the MDP that characterizes our problem
- While this assumption does not hold in many contexts, it does in many others
  - Organic chemistry
  - Puzzles
  - Quantum computing
  - Theorem proving
- Furthermore, it builds the foundation that we will use to explore model-free algorithms

Bellman Equation

- $v_\pi(s) = \mathbb{E}_\pi[G_t|S_t = s] = \mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} \,|S_t = s]$
- $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1}|S_t = s] = \mathbb{E}_\pi[R_{t+1}|S_t = s] + \gamma \mathbb{E}_\pi[G_{t+1}|S_t = s]$
- $\mathbb{E}_\pi[R_{t+1}|S_t = s] = \sum_r p(r|s)r = \sum_a \sum_r p(r,a|s)r = \sum_a \sum_r p(r|s,a)\pi(a|s)r$
- $\mathbb{E}_\pi[R_{t+1}|S_t = s] = \sum_a \pi(a|s) \sum_r p(r|s,a)r = \sum_a \pi(a|s)r(s,a)$
- $\mathbb{E}_\pi[G_{t+1}|S_t = s] = \sum_a \pi(a|s) \sum_{s'} p(s'|s,a)\mathbb{E}_\pi[G_{t+1}|S_{t+1} = s']$
- $\mathbb{E}_\pi[G_{t+1}|S_{t+1} = s'] = v_\pi(s')$
- $v_\pi(s) = \sum_a \pi(a|s)r(s,a) + \gamma \sum_a \pi(a|s) \sum_{s'} p(s'|s,a)v_\pi(s')$
- $v_\pi(s) = \sum_a \pi(a|s)(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\pi(s'))$

Policy Evaluation

- Estimate the expected future reward when following policy π
- From the Bellman equation we know that
  - $v_\pi(s) = \sum_a \pi(a|s)(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\pi(s'))$
- Given a policy π, what if we searched for a function V that satisfies the Bellman equation?
  - Will we have successfully evaluated π?
- If so, how should we search for V?
- Use the Bellman equation as an update rule
  - $V(s) = \sum_a \pi(a|s)(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, V(s'))$
- Will this converge to v_π?

Policy Evaluation: Convergence

- We know that the Bellman equation has a fixed point at v_π
- We can prove that the Bellman update is a contraction mapping
- We can prove that repeatedly updating V will converge to a unique fixed point
  - Bellman equation
- Convergence rate depends on γ
- Larger γ requires more iterations

Policy Evaluation: Convergence

$$v_\pi(s) = \sum_a \pi(a|s)(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\pi(s'))$$

$$v_\pi(s) = \sum_a \pi(a|s) r(s,a) + \gamma \sum_a \pi(a|s) \sum_{s'} p(s'|s,a)\, v_\pi(s')$$

$$\boldsymbol{v}^{\boldsymbol{\pi}} = \boldsymbol{r}^{\pi} + \gamma \boldsymbol{P}^{\boldsymbol{\pi}} \boldsymbol{v}^{\pi}$$

- Matrices:
  - $\boldsymbol{v}^{\boldsymbol{\pi}}$ is a vector of values for each state, size $|\mathcal{S}|$
  - $\boldsymbol{r}^{\boldsymbol{\pi}}$ is a vector of rewards for each state, size $|\mathcal{S}|$
  - $\boldsymbol{P}^{\boldsymbol{\pi}}$ is a matrix of transition probabilities for each pair of states, size $|\mathcal{S}|$x $|\mathcal{S}|$

Policy Evaluation: Convergence

- Define T as the Bellman backup operator:

$$T(\boldsymbol{v}) \coloneqq \boldsymbol{r}^{\pi} + \gamma \boldsymbol{P}^{\boldsymbol{\pi}} \boldsymbol{v}$$

- We know that there exists a fixed point

$$T(\boldsymbol{v}^{\boldsymbol{\pi}}) = \boldsymbol{v}^{\pi}$$

- The infinity norm:

$$\left|\left|\boldsymbol{x}\right|\right|_{\infty} \coloneqq \max_{\mathrm{i}} |\mathrm{x_i}|$$

- If:

$$\left|\left|T(\boldsymbol{u}) - T(\boldsymbol{v})\right|\right|_{\infty} \leq \gamma \left|\left|\boldsymbol{u} - \boldsymbol{v}\right|\right|_{\infty}$$

$$\text{for } \gamma \in [0,1)$$

- Then T is a contraction mapping
- Banach-Caccioppoli fixed point theorem proves that repeated applications of T will converge to a unique fixed point (i.e. $\boldsymbol{v}^{\pi}$).

Policy Evaluation: Convergence

$$\left|\left|T(\boldsymbol{u}) - T(\boldsymbol{v})\right|\right|_\infty = \left|\left|\boldsymbol{r}^\pi + \gamma \boldsymbol{P}^\pi \boldsymbol{u} - (\boldsymbol{r}^\pi + \gamma \boldsymbol{P}^\pi \boldsymbol{v})\right|\right|_\infty$$
$$= \left|\left|\gamma \boldsymbol{P}^\pi \boldsymbol{u} - \gamma \boldsymbol{P}^\pi \boldsymbol{v}\right|\right|_\infty$$
$$= \gamma \left|\left|\boldsymbol{P}^\pi (\boldsymbol{u} - \boldsymbol{v})\right|\right|_\infty$$
$$\leq \gamma \left|\left|\boldsymbol{P}^\pi \left|\left|\boldsymbol{u} - \boldsymbol{v}\right|\right|_\infty\right|\right|_\infty$$ **// $\boldsymbol{P}^\pi$** is a matrix of transition probabilities, sums to 1
$$\leq \gamma \left|\left|\boldsymbol{u} - \boldsymbol{v}\right|\right|_\infty$$

Policy Evaluation

Algorithm Policy Evaluation

Inputs:
    π: Policy to be evaluated
    V: Initial value function. Value of terminal state must be zero
    θ: Termination threshold

$\Delta \leftarrow \inf$
while $\Delta > \theta$ **do**
    for $s \in \mathcal{S}$ **do**
        $v \leftarrow V(s)$
        $V(s) \leftarrow \sum_a \pi(a|s)(r(s,a) + \gamma \sum_{s'} p(s'|s,a)V(s'))$ ▷ Bellman equation
        $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
    end for
end while
return V ▷ Approximation of v_π

Policy Evaluation AI Farm

- Policy: Uniform random
- AI Farm took 4576 iterations to converge with $\gamma = 1$
  - $\gamma = 0.99$ takes 1345 iterations
  - $\gamma = 0.9$ takes 191 iterations
- What if policy said to always go up?
  - $\gamma = 1$
  - $\gamma < 1$

Policy Improvement

- We have evaluated policy π, how can we find a better policy?
- $\pi' \geq \pi$ if and only if $v_{\pi'}(s) \geq v_{\pi}(s)$ for all $s \in \mathcal{S}$
- We set the policy to be greedy with respect to v_{π}
- $\pi'(s) = \underset{a}{\operatorname{argmax}}(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_{\pi}(s'))$
- Will this policy always be the same or better than the previous one?

Policy Improvement Theorem

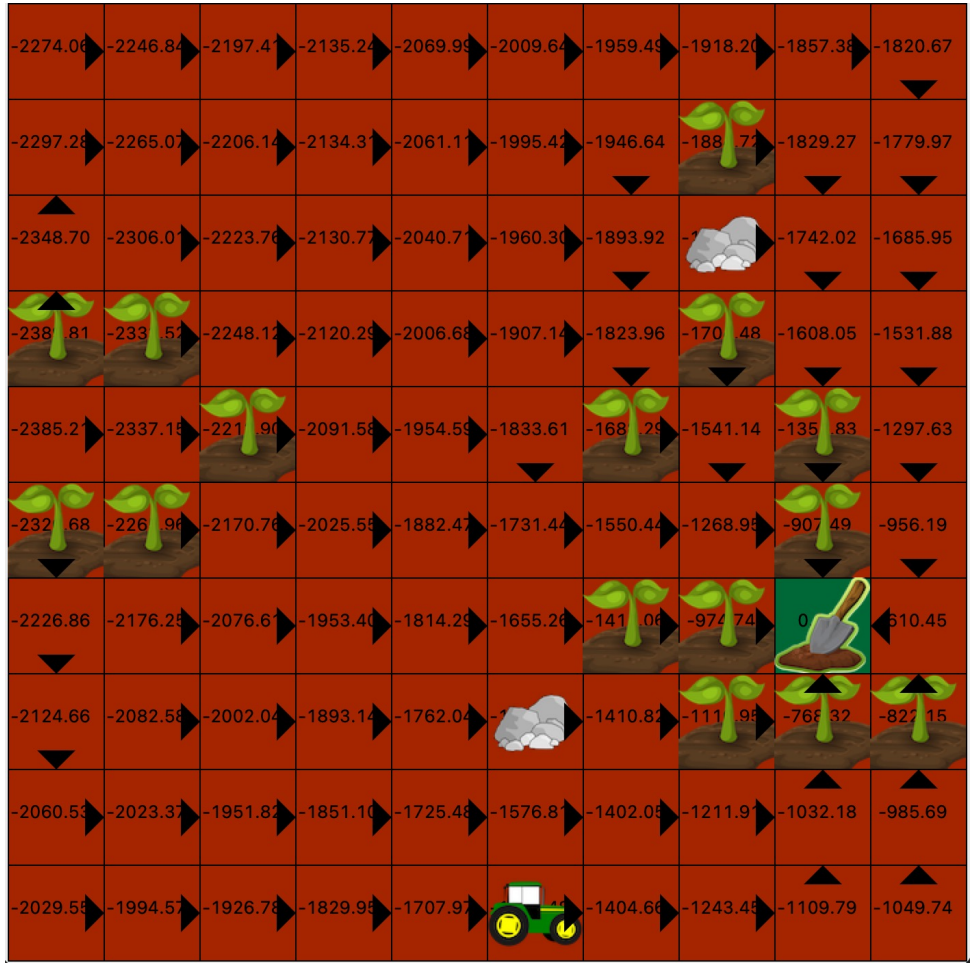
- Let's first look at one step
- $v_\pi(s) = \sum_a \pi(a|s)(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\pi(s'))$
- $\pi'(s) = \underset{a}{\text{argmax}}(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_\pi(s'))$
- $q_\pi\big(s, \pi'(s)\big) = r(s,a) + \gamma \sum_{s'} p(s'|s,\pi'(s))\, v_\pi(s') \geq v_\pi(s)$
  - For all states s

Policy Improvement Theorem

$$
\begin{aligned}
v_\pi(s) &\le q_\pi(s, \pi'(s)) \\
&= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)] && \text{(by (4.6))} \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\
&\le \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] && \text{(by (4.7))} \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma \mathbb{E}[R_{t+2} + \gamma v_\pi(S_{t+2}) | S_{t+1}, A_{t+1} = \pi'(S_{t+1})] \mid S_t = s] \\
&= \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s\right] \\
&\le \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3}) \mid S_t = s\right] \\
&\vdots \\
&\le \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \mid S_t = s\right] \\
&= v_{\pi'}(s).
\end{aligned}
$$

Policy Improvement

- Better than original uniform random policy
- Still not optimal



When acting greedily with respect to v_π

Policy Iteration

- Policy improvement improves a policy π and obtains a new policy π' such that, $\pi' \geq \pi$
- If $\pi' = \pi$, then $v_{\pi'} = v_{\pi}$
- Therefore, $v_{\pi'}(s) = \max_a (r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_{\pi'}(s'))$
- This is the same as the Bellman optimality equation!
- Does this mean $\pi' = \pi_*$ and $v_{\pi'} = v_*$?
- There is a proof showing that the Bellman optimality equation is a unique fixed point
  - Similar to that of the Bellman equation

Policy Iteration

Algorithm Policy Iteration

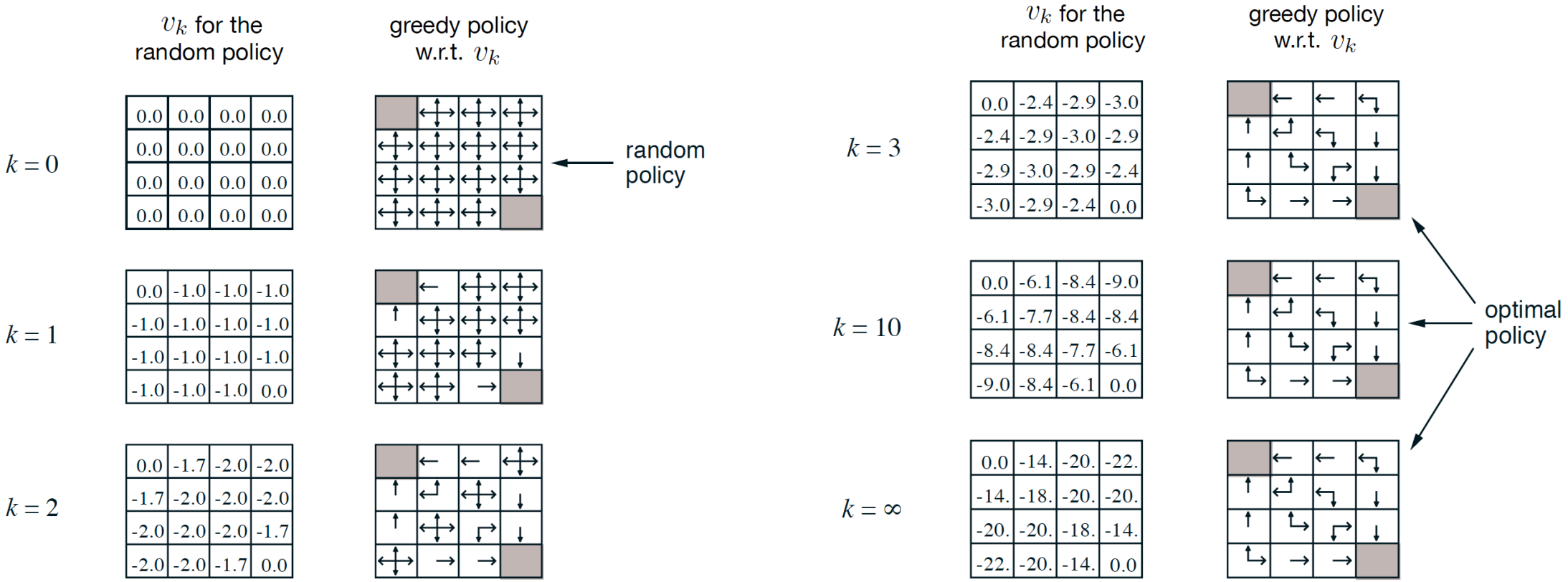
Inputs:
    π: Initial Policy
    V: Initial value function. Value of terminal state must be zero
    θ: Termination threshold

while has not converged **do**
    $V = \text{Policy\_Evaluation}(\pi, V, \theta)$
    $\pi = \text{Policy\_Improvement}(V)$
end while
return π ▷ Approximation of $\pi*$

Policy Iteration: AI Farm

```
(drl) forestagostinelli@Forests-MacBook-Pro Farm_Grid_World % pytho
n run_policy_iteration.py --map maps/map1.txt --wait 1.0 --wait_eva
l 0.0 --rand_right 0.0
```

Do We Have to Wait For Policy Evaluation to Converge?

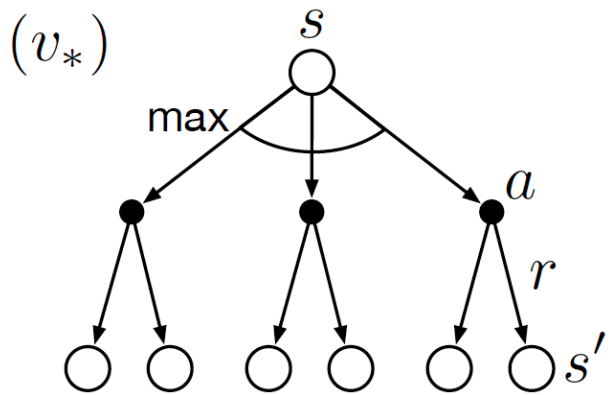


Modified Policy Iteration

- We do not have to run policy evaluation to convergence.
- We can stop after reaching some threshold
- We can stop after k iterations
  - $k = 1$ is the same as value iteration

Bellman Optimality Equation

- We can write v_*, the value of an optimal policy, π_*, in terms of itself
- $v_*(s) = \max_a(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_*(s'))$



Backup diagram for v^*

Value Iteration

- Find the optimal value function
- Recall the Bellman optimality equation
  - $v_*(s) = \max_a(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, v_*(s'))$
- Use this as an update rule
  - $V(s) = \max_a(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, V(s'))$
- Combines policy evaluation and policy improvement into one step
- There is a proof similar to that of policy evaluation to prove that repeatedly updating V will converge to a unique fixed point
  - Bellman optimality equation

Value Iteration

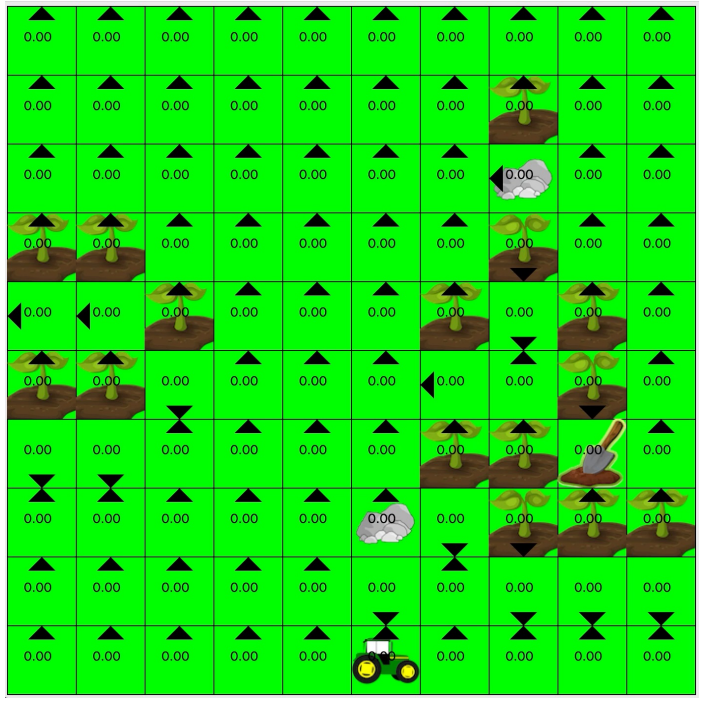
Algorithm Value Iteration

Inputs:
    V: Initial value function. Value of terminal state must be zero
    θ: Termination threshold

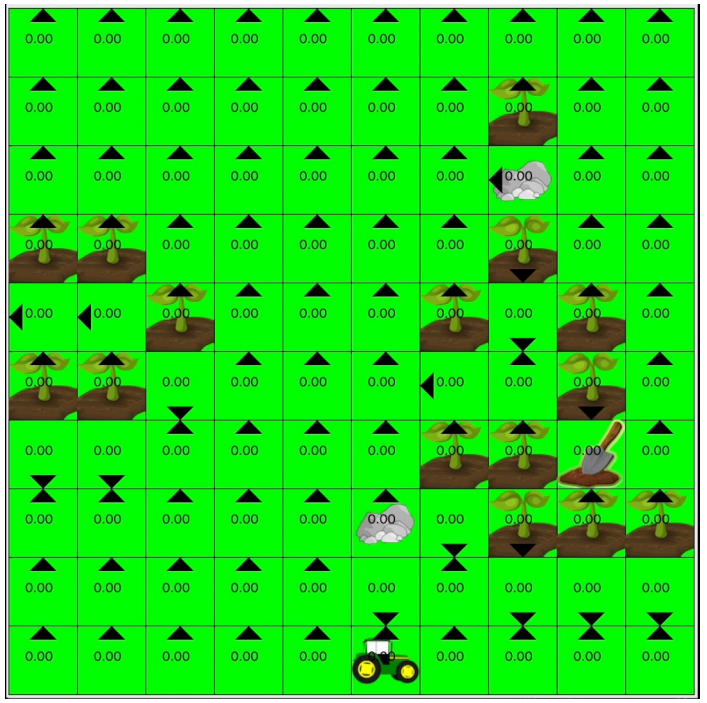
procedure VALUE_ITERATION(V, θ, γ)
    $\Delta \leftarrow \text{inf}$
    while $\Delta > \theta$ **do**
        $\Delta \leftarrow 0$
        for $s \in \mathcal{S}$ **do**
            $v \leftarrow V(s)$
            $V(s) \leftarrow \max_a(r(s,a) + \gamma \sum_{s'} p(s'|s,a)V(s'))$
            $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
        end for
    end while
    return V ▷ Approximation of v_*
end procedure

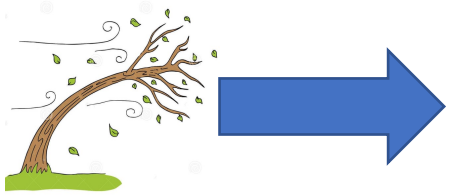
Value Iteration: AI Farm

$p = 0$

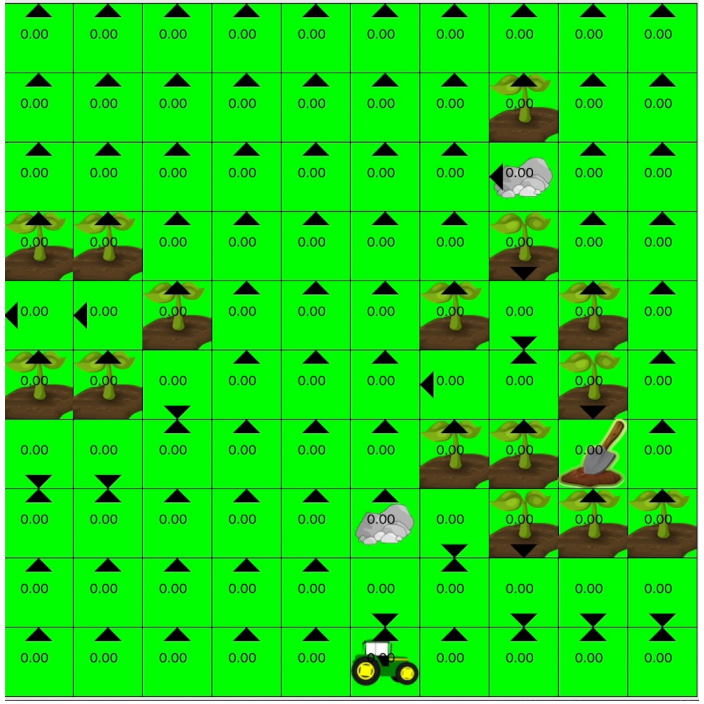


$p = 0.1$





$p = 0.5$



Asynchronous Policy Iteration

- We do not have to update all states at every iteration
- Prioritize states based on bellman error
  - Difference between current and updated estimation of state value
- Prioritize states based on relevance to the agent's experience
- To converge, must not completely ignore states
  - i.e. must continue to update all states, just not on every iteration

Summary

- **Policy Evaluation**: Uses Bellman equation as an update rule

$$V(s) = \sum_a \pi(a|s)(r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, V(s'))$$

- **Policy Improvement**: Behave greedily with respect to value function

$$\pi'(s) = \operatorname*{argmax}_a (r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, V(s'))$$

- **Policy Iteration**: Iterate between policy evaluation and policy improvement until convergence
- **Value Iteration**: Uses Bellman optimality equation as an update rule

$$V(s) = \max_a (r(s,a) + \gamma \sum_{s'} p(s'|s,a)\, V(s'))$$